

Note on Hyperbolic Rotations

Shohruh MIRYUSUPOV

2024-01-15

Contents

1	Hyperbolic Downdating	1
1.1	Hyperbolic rotations	1
1.1.1	Algorithm HY	1
1.2	HY Algorithm Lower Cholesky Factor	2
1.3	Algorithm HC	3
1.3.1	Algorithm HC Upper	3
1.3.2	Algorithm HC Lower	3
1.4	Algorithm A	3
1.4.1	Algorithm A Upper	3
1.4.2	Algorithm A Lower	5
2	Implicit Upper Triangular times vector	6
2.1	Implicit Lower Triangular times vector (HY)	6
2.2	Implicit Lower Triangular times vector (HY) Unrolled version	6
2.3	Implicit Lower Triangular times vector (Algorithm A)	6
2.4	Implicit Lower Triangular times vector (HC)	9
3	Numerical experiments	10
3.1	Benchmark description	10
3.1.1	Timining benchmarks	10
3.1.2	Complexity Ration	10
3.2	Upper triangular matrix	10
3.3	Lower triangular matrix	12
3.4	Cholvec Algorithm	15
3.4.1	Uppertriangular case	15
3.4.2	Lowertriangular case	16
3.5	Rank M Cholesky Factor Update	18

1 Hyperbolic Downdating

Definition 1.1. A pseudo-orthogonal matrix $P \in \mathbf{R}^{n,n}$ with respect to S is a matrix that satisfies:

$$P^T S P = S \tag{1}$$

where $S = \mathbf{diag}(\pm 1)$. If $S = \mathbb{I}$, then S is an orthogonal matrix.

Now let $S = \begin{pmatrix} \mathbb{I}_n & \\ & -1 \end{pmatrix}$, then

$$\begin{bmatrix} R^T & z \end{bmatrix} S \begin{bmatrix} R \\ z^T \end{bmatrix} = R^T R - z z^T$$

If we choose a proper orthogonal matrix P such that

$$P \begin{bmatrix} R \\ z^T \end{bmatrix} = \begin{bmatrix} D \\ 0^T \end{bmatrix}$$

D is an upper triangular Cholesky factor, then

$$R^T R - z z^T = D^T D$$

1.1 Hyperbolic rotations

. Suppose a matrix $H \in \mathbf{R}^{n \times n}$ is

$$H(k, l) = \begin{bmatrix} 1 & & & & \\ \cdots & \cosh \theta & \cdots & -\sinh \theta & \cdots \\ & \vdots & & \vdots & \\ \cdots & -\sinh \theta & \cdots & \cosh \theta & \cdots \\ & \vdots & & \vdots & 1 \end{bmatrix}, \quad \theta \in \mathbf{R}$$

then H is pseudo-orthogonal matrix with respect to $S = \mathbf{diag}(1, \dots, 1, -1, 1, \dots, 1)$.

$$\cosh \theta = \frac{x_k}{\sqrt{x_k^2 - x_l^2}}, \quad \sinh \theta = \frac{x_l}{\sqrt{x_k^2 - x_l^2}}$$

1.1.1 Algorithm HY

For $i = 1, 2, \dots, n$ we determine the hyperbolic rotations $P_i \in \mathbf{R}^{(n+1) \times (n+1)}$ such that

$$P_n \dots P_1 \begin{bmatrix} R \\ z^T \end{bmatrix} = \begin{bmatrix} D \\ 0^T \end{bmatrix}$$

Teh, the i -th step is

$$P_i \begin{bmatrix} R^{(i-1)} \\ (z^{(i-1)})^T \end{bmatrix} = \begin{bmatrix} R^{(i)} \\ (z^{(i)})^T \end{bmatrix}$$

and

$$R^{(i)} = \begin{bmatrix} D_1 \\ \vdots \\ D_i \\ R_{i+1} \\ \vdots \\ R_n \end{bmatrix}$$

where

$$R = \begin{bmatrix} R_1 \\ \vdots \\ R_n \end{bmatrix} \quad \text{and} \quad D = \begin{bmatrix} D_1 \\ \vdots \\ D_n \end{bmatrix}$$

Then

$$P_i = H(i, n+1, r_{ii}, z_i^{(i-1)})$$

and

$$\cosh \theta_i = \frac{r_{ii}}{\sqrt{r_{ii}^2 - (z_i^{(i-1)})^2}}, \quad \sinh \theta_i = \frac{z_i^{(i-1)}}{\sqrt{r_{ii}^2 - (z_i^{(i-1)})^2}}$$

If $\frac{z_i^{(i-1)}}{|r_{ii}|} < 1$, then $R^T R - z z^T$ is PD.

1.2 HY Algorithm Lower Cholesky Factor

Algorithm 1 HY algorithm computes the downdated Cholesky Upper factor $D \equiv (d_{ij}) \in \mathbf{R}^{n,n}$ such that $D^T D = R^T R - z z^T$

Input: Initial values R, z .

Output: Downdated Cholesky factor D .

```

for  $i = 1$  to  $n$  do
  if  $r_{ii}^2 - (z_i^{(i-1)})^2 \leq 0$  then
    STOP
  else
    Update  $ch_i = \frac{r_{ii}}{\sqrt{r_{ii}^2 - (z_i^{(i-1)})^2}}$ 
    Update  $sh_i = \frac{z_i^{(i-1)}}{\sqrt{r_{ii}^2 - (z_i^{(i-1)})^2}}$ 
    Update  $d_{ii} = \sqrt{r_{ii}^2 - (z_i^{(i-1)})^2}$ 
  end if
  for  $j = i + 1$  to  $n$  do
     $d_{ij} = ch_i \cdot r_{ij} - sh_i \cdot z_j^{(i-1)}$ 
     $z_j^{(i)} = -sh_i \cdot r_{ij} + ch_i \cdot z_j^{(i-1)}$ 
  end for
end for

```

Algorithm 2 HY algorithm computes the downdated Cholesky Lower factor $D \equiv (d_{ij}) \in \mathbf{R}^{n,n}$ such that $DD^T = LL^T - z z^T$

Input: Initial values L, z .

Output: Downdated Cholesky factor D .

```

for  $i = 1$  to  $n$  do
  if  $L_{ii}^2 - (z_i^{(i-1)})^2 \leq 0$  then
    STOP
  else
    Update  $ch_i = \frac{L_{ii}}{\sqrt{L_{ii}^2 - (z_i^{(i-1)})^2}}$ 
    Update  $sh_i = \frac{z_i^{(i-1)}}{\sqrt{L_{ii}^2 - (z_i^{(i-1)})^2}}$ 
    Update  $d_{ii} = \sqrt{L_{ii}^2 - (z_i^{(i-1)})^2}$ 
  end if
  for  $j = i + 1$  to  $n$  do
     $d_{ji} = ch_i \cdot L_{ji} - sh_i \cdot z_j^{(i-1)}$ 
     $z_j^{(i)} = -sh_i \cdot L_{ji} + ch_i \cdot z_j^{(i-1)}$ 
  end for
end for

```

1.3 Algorithm HC

1.3.1 Algorithm HC Upper

This algorithm was proposed by Chambers (1971) [1].

Algorithm 3 HC algorithm computes the downdated Cholesky factor $D \equiv (d_{ij}) \in \mathbf{R}^{n,n}$ such that $D^T D = R^T R - z z^T$

Input: Initial values R, z .

Output: Downdated Cholesky factor D .

for $i = 1$ to n **do**

$$s_i = \frac{z_i}{r_{ii}}$$

if $1 - s_i^2 \leq 0$ **then**

STOP

else

Update $d_i = c_i r_{ii}$

end if

for $j = i + 1$ to n **do**

$$d_{ij} = \frac{r_{ij} - s_i \cdot z_j^{(i-1)}}{c_i}$$

$$z_j^{(i)} = -s_i \cdot d_{ij} + c_i \cdot z_j^{(i-1)}$$

end for

end for

1.3.2 Algorithm HC Lower

Algorithm 4 HC algorithm computes the downdated Cholesky Lower factor $D \equiv (d_{ij}) \in \mathbf{R}^{n,n}$ such that $DD^T = LL^T - z z^T$, using hyperbolic rotations for stability.

Input: Initial values L, z .

Output: Downdated Cholesky factor D .

for $i = 1$ to n **do**

Calculate scaling factor $s_i = \frac{z_i^{(updated)}}{L_{ii}}$.

if $1 - s_i^2 \leq 0$ **then**

Raise error: Downdating failed due to the square root of a negative number encountered.

else

Calculate hyperbolic cosine $c_i = \sqrt{1 - s_i^2}$.

Update diagonal element $d_{ii} = c_i \cdot L_{ii}$.

for $j = i + 1$ to n **do**

$$\text{Update } d_{ji} = \frac{L_{ji} - s_i \cdot z_j^{(updated)}}{c_i}.$$

$$\text{Update } z_j^{(updated)} = -s_i \cdot d_{ji} + c_i \cdot z_j^{(updated)}.$$

end for

end if

end for

1.4 Algorithm A

1.4.1 Algorithm A Upper

The algorithm A is more stable then Alogirhtm HY. This is version allows to compute upper triangular Cholesky factor D .

1.4.2 Algorithm A Lower

This is version allows to compute upper triangular Cholesky factor D .

Algorithm 5 Algorithm A computes the downdated Cholesky factor $D \equiv (d_{ij}) \in \mathbf{R}^{n,n}$ such that $D^T D = R^T R - zz^T$ without building D

Input: Initial values R, z .

Output: Downdated Cholesky factor D .

$\alpha_0 = 1, z \equiv \bar{z}^{(0)}$

for $i = 1$ to n **do**

$a_i = \frac{\bar{z}_i^{i-1}}{r_{ii}}$

$\alpha_i = \alpha_{i-1} - a_i^2$

$\beta_i = \sqrt{\alpha_i}$

$d_{ii} = \frac{\beta_i}{\beta_{i-1}} r_{ii}$

for $j = i + 1$ to n **do**

$\bar{z}_j^{(i)} = \bar{z}_j^{(i-1)} - a_i r_{ij}$

$d_{ij} = \frac{\beta_i}{\beta_{i-1}} r_{ij} - \frac{a_i}{\beta_{i-1} \beta_i} \bar{z}_j^{(i)}$

end for

end for

Algorithm 6 Algorithm A for Lower Triangular Matrix Downdating

Input: Lower triangular matrix $L \in \mathbf{R}^{n \times n}$, vector $z \in \mathbf{R}^n$.

Output: Downdated Cholesky factor $D \in \mathbf{R}^{n \times n}$.

Initialize D as a zero matrix of the same dimensions as L .

Initialize $\alpha \in \mathbf{R}^{n+1}$ with $\alpha_0 = 1$.

Copy z into $\bar{z}$.

for $i = 1$ to n **do**

Calculate $a_i = \frac{\bar{z}_i}{L_{ii}}$.

Update $\alpha_{i+1} = \alpha_i - a_i^2$.

if $\alpha_{i+1} \leq 0$ **then**

Raise an error indicating downdating cannot proceed.

end if

Calculate $\beta_i = \sqrt{\alpha_{i+1}}$.

Set $\beta_{i-1} = \sqrt{\alpha_i}$ if $i > 1$, otherwise $\beta_{i-1} = 1$.

Update $d_{ii} = \left(\frac{\beta_i}{\beta_{i-1}} \right) \cdot L_{ii}$.

for $j = i + 1$ to n **do**

Update $\bar{z}_j = \bar{z}_j - a_i \cdot L_{ji}$.

Update $d_{ji} = \left(\frac{\beta_i}{\beta_{i-1}} \right) \cdot L_{ji} - \left(\frac{a_i}{\beta_{i-1} \beta_i} \right) \cdot \bar{z}_j$.

end for

end for

2 Implicit Upper Triangular times vector

Given D an upper triangular Cholesky factor, we want to compute Dv without explicitly constructing D . Note

$$\begin{bmatrix} \nu & 0 \end{bmatrix} P_n \dots P_1 \begin{bmatrix} R \\ z^T \end{bmatrix} = \begin{bmatrix} \nu & 0 \end{bmatrix} \begin{bmatrix} D \\ 0^T \end{bmatrix}$$

Then

$$\begin{bmatrix} \nu & 0 \end{bmatrix} P_n \dots P_1 = \begin{bmatrix} D\nu & * \end{bmatrix}$$

Algorithm 7 HY Downdating Algorithm with Vector Multiplication: Computes the downdated vector w from the Cholesky factor R and vectors z, v , ensuring $w = (R^T R - zz^T)v$.

Input: Initial values R, z, v .

Output: Downdated vector w .

for $i = 1$ to n **do**

if $r_{ii}^2 - z_i^2 \leq 0$ **then**

 Raise an error indicating downdating failed due to the square root of a negative number encountered.

end if

 Compute $ch_i = \frac{r_{ii}}{\sqrt{r_{ii}^2 - z_i^2}}$ and $sh_i = \frac{z_i}{\sqrt{r_{ii}^2 - z_i^2}}$.

 Initialize $w_i = \sqrt{r_{ii}^2 - z_i^2} \cdot v_i$.

for $j = i + 1$ to n **do**

 Compute intermediate value $w_{\text{temp},i} = (ch_i \cdot r_{ij} - sh_i \cdot z_j) \cdot v_j$.

 Update $z_j = -sh_i \cdot r_{ij} + ch_i \cdot z_j$.

 Accumulate $w_i = w_i + w_{\text{temp},i}$.

end for

end for

Algorithm 8 HC Downdating Algorithm with Vector Multiplication for Hyperbolic Rotations. Computes the downdated vector w from the Cholesky factor R and vectors z, v , ensuring $w = (R^T R - zz^T)v$, without explicitly constructing D .

Input: Cholesky factor R , downdating vector z , multiplication vector v .

Output: Downdated vector w after applying the downdate and multiplication.

for $i = 1$ to n **do**

 Calculate scaling factor $s_i = \frac{z_i^{(\text{updated})}}{r_{ii}}$.

if $1 - s_i^2 \leq 0$ **then**

 Raise error due to square root of a negative number.

end if

 Compute cosine $c_i = \sqrt{1 - s_i^2}$.

 Initialize downdated vector element $w_i = c_i \cdot r_{ii} \cdot v_i$.

for $j = i + 1$ to n **do**

 Compute $a = \frac{r_{ij} - s_i \cdot z_j^{(\text{updated})}}{c_i}$.

 Update $z_j^{(\text{updated})} = -s_i \cdot a + c_i \cdot z_j^{(\text{updated})}$.

 Accumulate $w_i = w_i + a \cdot v_j$.

end for

end for

2.1 Implicit Lower Triangular times vector (HY)

2.2 Implicit Lower Triangular times vector (HY) Unrolled version

2.3 Implicit Lower Triangular times vector (Algorithm A)

Algorithm 9 Algorithm Av: Downdating with Vector Multiplication based on Algorithm A. Computes the downdated vector w from the Cholesky factor R and vectors z , v , ensuring $w = (R^T R - zz^T)v$, without explicitly constructing D .

Input: Cholesky factor R , downdating vector z , multiplication vector v .

Output: Resulting vector w after downdating and multiplication.

for $i = 1$ to n **do**

 Compute scaling factor $a_i = \frac{z_i}{r_{ii}}$.

 Update $\alpha_{i+1} = \alpha_i - a_i^2$.

if $\alpha_{i+1} \leq 0$ **then**

 Raise error due to non-positive downdating condition.

end if

 Compute $\beta_i = \sqrt{\alpha_{i+1}}$ and $\beta_{i-1} = \sqrt{\alpha_i}$ (or 1 if $i = 1$).

 Update downdated vector element $w_i = \frac{\beta_i}{\beta_{i-1}} \cdot r_{ii} \cdot v_i$.

for $j = i + 1$ to n **do**

 Update $z_j = z_j - a_i \cdot r_{ij}$.

 Compute downdate adjustment $d_{ij} = \left(\frac{\beta_i}{\beta_{i-1}} \cdot r_{ij} \right) - \left(\frac{a_i}{\beta_{i-1} \cdot \beta_i} \cdot z_j \right)$.

 Accumulate $w_i = w_i + d_{ij} \cdot v_j$.

end for

end for

Algorithm 10 HY Algorithm for Downdating Cholesky Factor with Identity as Matrix Structure

Input: Cholesky factor R , vector z , scaling factor α , vector v .

Output: Vector w resulting from the downdating process.

Initialize n to the number of rows in R .

Copy z to z_{updated} .

Initialize w as a zero vector with the same shape as v .

Set r_{ii} to 1.

for $i = 1$ to n **do**

 Compute $temp = \sqrt{1 + \alpha \cdot z_{\text{updated}}[i]^2}$.

if $temp \leq 0$ **then**

 Raise an error indicating the update failed due to square root of a negative number.

end if

 Update $ch[i] = \frac{1}{temp}$.

 Update $sh[i] = z_{\text{updated}}[i] \cdot ch[i]$.

 Increment $w[i] += temp \cdot v[i]$.

if $i + 1 < n$ **then**

 Increment $w[i + 1 :] += \alpha \cdot sh[i] \cdot z_{\text{updated}}[i + 1 :] \cdot v[i]$.

 Scale $z_{\text{updated}}[i + 1]* = ch[i]$.

end if

end for

Algorithm 11 Unrolled HY Algorithm for Downdating Cholesky Factor

Input: Cholesky factor R , vector z , scaling factor α , vector v , number of unrolled steps $nsteps$.

Output: Vector w resulting from the downdating process.

Initialize n to the number of rows in R .

Initialize w as a zero vector with the same shape as v .

$i = 0$

while $i < n$ **do**

for $step$ in 0 to $\min(nsteps - 1, n - i - 1)$ **do**

$idx = i + step$ {Current index for this step}

 Compute $temp = \sqrt{1 + \alpha \cdot z[idx]^2}$.

if $temp \leq 0$ **then**

 Raise an error indicating the update failed due to square root of a negative number.

end if

 Increment $w[idx] += temp \cdot v[idx]$.

if $idx + 1 < n$ **then**

 Increment $w[idx + 1 :] += \alpha \cdot z[idx] \cdot z[idx + 1 :] \cdot v[idx] / temp$.

 Scale $z[idx + 1 :] *= 1 / temp$.

end if

end for

$i += nsteps$ {Increment by $nsteps$ for the next iteration}

end while

Algorithm 12 Alga Cholesky Update for Vector

Input: Vector z , vector v , scaling factor α .

Output: Updated vector w .

Initialize n to the length of vector v .

Initialize vector λ of length $n + 1$ with $\lambda[0] = 1$.

Initialize w as a zero vector with the same shape as v .

for $i = 0$ to $n - 1$ **do**

 Update $\lambda[i + 1] = \lambda[i] + \alpha \cdot z[i]^2$.

 Compute $\beta_i = \sqrt{\lambda[i + 1]}$.

 Compute $\beta_{i-1} = \sqrt{\lambda[i]}$.

 Compute $s = \beta_{i-1} \cdot \beta_i$.

 Update $w[i] += \frac{\beta_i}{\beta_{i-1}} \cdot v[i]$.

for $j = i + 1$ to n **do**

 Update $w[j] += \frac{\alpha \cdot (z[i] \cdot z[j])}{s} \cdot v[i]$.

end for

end for

2.4 Implicit Lower Triangular times vector (HC)

Algorithm 13 HC Cholesky Update for Lower Triangular Matrix

Input: Vector z , vector v , scaling factor α .

Output: Updated vector w .

Initialize n to the length of vector v .

Initialize w as a zero vector with the same shape as v .

for $i = 0$ to $n - 1$ **do**

 Compute $s = 1 + \alpha \cdot z[i]^2$.

if $\alpha = -1$ and $s \leq 0$ **then**

 Raise an error indicating downdating failed due to square root of a negative number.

end if

 Compute $c = \sqrt{s}$.

 Update $w[i] += c \cdot v[i]$.

for $j = i + 1$ to n **do**

 Compute $d_{ij} = \frac{\alpha \cdot z[i] \cdot z[j]}{c}$.

 Update $w[j] += d_{ij} \cdot v[i]$.

 Update $z[j] = -z[i] \cdot d_{ij} + c \cdot z[j]$.

end for

end for

3 Numerical experiments

3.1 Benchmark description

3.1.1 Timing benchmarks

For each method and matrix size, the benchmarking is performed with N_{rep} repetitions to average out runtime variability, using a slightly perturbed version of z to simulate updates or downdates. The experiment records the average execution time for each update method and matrix size, consolidating these into a table, it is organized as the summary of the execution times across different algorithms and matrix sizes, enabling an easy comparison of performance scalability and efficiency as the problem size increases.

3.1.2 Complexity Ration

The complexity ratio of a dataset provides a quantified measure of the relative change in data over successive intervals, essentially capturing the growth, decay, or volatility within the dataset. Given a sequence of values $x_1, x_2, \dots, x_n$, the complexity ratio for each x_i where $i \in 2, 3, \dots, n$ is computed using the percentage change formula:

$$CR_i = \frac{x_i - x_{i-1}}{x_i} + 1$$

This formula calculates the rate of change between consecutive values in the dataset.

3.2 Upper triangular matrix

This experiment is designed to compare the correctness of different algorithms for downdating the Cholesky decomposition of a positive definite matrix. Specifically, it focuses on the Cholesky factor R of a matrix, downdated by a rank-1 update defined by a vector z . The algorithms tested include:

- Numpy’s Cholesky Decomposition: Utilized as a baseline for comparison, this method involves directly computing the Cholesky decomposition of the downdated matrix $R^T R - zz^T$ using NumPy’s built-in `np.linalg.cholesky` function.
- HY Algorithm: An implementation specifically designed for downdating the Cholesky decomposition, handling the "upper" triangular form of the matrix. This method is expected to preserve the Cholesky factor’s upper triangular property while adjusting for the subtraction of zz^T .
- HC Algorithm: Another downdating approach that employs hyperbolic rotations (or cosine adjustments) to update the Cholesky factor. Like the HY algorithm, it operates on the "upper" triangular matrix, adjusting it to reflect the downdate.
- Algorithm A: A stable method for Cholesky downdating.

The experiment proceeds as follows:

1. Generate a symmetric positive definite matrix by creating a random matrix R , multiplying it by its transpose, and adding a scalar to ensure positiveness. Then, compute its Cholesky decomposition to obtain the upper triangular matrix.
2. Compute the Cholesky factor D of the downdated matrix $R^T R - zz^T$, where z is a downdating vector. using NumPy’s Cholesky decomposition as a reference.
3. Apply the HY, HC, and Algorithm A approaches to downdate the Cholesky factor R directly, without recomputing the entire decomposition from the downdated matrix.

Table 1: Numpy Cholesky matrix

3.730666	1.706841	2.705946	2.0659233	1.2701729
0.000000	2.248513	1.186822	0.8484672	1.3806639
0.000000	0.000000	1.131258	0.8558580	1.0243914

0.000000	0.000000	0.000000	1.8559527	0.2422083
0.000000	0.000000	0.000000	0.0000000	2.0862363

Table 2: HY matrix

3.730666	1.706841	2.705946	2.0659233	1.2701729
0.000000	2.248513	1.186822	0.8484672	1.3806639
0.000000	0.000000	1.131258	0.8558580	1.0243914
0.000000	0.000000	0.000000	1.8559527	0.2422083
0.000000	0.000000	0.000000	0.0000000	2.0862363

Table 3: HC matrix

3.730666	1.706841	2.705946	2.0659233	1.2701729
0.000000	2.248513	1.186822	0.8484672	1.3806639
0.000000	0.000000	1.131258	0.8558580	1.0243914
0.000000	0.000000	0.000000	1.8559527	0.2422083
0.000000	0.000000	0.000000	0.0000000	2.0862363

Table 4: Algorithm A matrix

3.730666	1.706841	2.705946	2.0659233	1.2701729
0.000000	2.248513	1.186822	0.8484672	1.3806639
0.000000	0.000000	1.131258	0.8558580	1.0243914
0.000000	0.000000	0.000000	1.8559527	0.2422083
0.000000	0.000000	0.000000	0.0000000	2.0862363

Table 5: Numpy Cholesky matrix

3.730666	1.706841	2.705946	2.0659233	1.2701729
0.000000	2.248513	1.186822	0.8484672	1.3806639
0.000000	0.000000	1.131258	0.8558580	1.0243914
0.000000	0.000000	0.000000	1.8559527	0.2422083
0.000000	0.000000	0.000000	0.0000000	2.0862363

Table 6: HY matrix

3.761445	1.771191	2.819257	2.1282885	1.3161464
0.000000	2.282174	1.290271	0.8962001	1.4074845
0.000000	0.000000	1.234150	0.8651327	0.9531466
0.000000	0.000000	0.000000	1.8626011	0.2731702
0.000000	0.000000	0.000000	0.0000000	2.1162630

Table 7: HC matrix

3.761445	1.771191	2.819257	2.1282885	1.3161464
0.000000	2.282174	1.290271	0.8962001	1.4074845
0.000000	0.000000	1.234150	0.8651327	0.9531466
0.000000	0.000000	0.000000	1.8626011	0.2731702
0.000000	0.000000	0.000000	0.0000000	2.1162630

Table 8: Algorithm A matrix

3.761445	1.692874	2.6838029	2.0490179	1.2597791
0.000000	2.340857	0.9486911	0.6869462	1.2417786
0.000000	0.000000	1.7416439	-0.0201998	0.1100837
0.000000	0.000000	0.0000000	2.2090148	-0.4522263
0.000000	0.000000	0.0000000	0.0000000	2.4143836

3.3 Lower triangular matrix

In this subsequent experiment, the focus shifts to evaluating the performance of downdating algorithms specifically tailored for lower triangular matrices. The aim is to ascertain their accuracy in modifying the Cholesky factor of a symmetric positive definite matrix, following a rank-1 update defined by a vector z , for matrices where the Cholesky factor is initially lower triangular. The methods under comparison include:

- Numpy’s Cholesky Decomposition: Serves as the benchmark, calculating the Cholesky decomposition of the downdated matrix $LL^T - zz^T$ to obtain a lower triangular matrix.
- Specialized Downdating Algorithms: Adaptations of the HY, HC, and Algorithm A methods, now optimized for the context of lower triangular matrices. These variations aim to adjust the Cholesky factor while accommodating the subtraction of zz^T , thus maintaining the lower triangular structure.

We continue the same steps as in the case of upper triangular matrices.

Table 9: Numpy Cholesky matrix

4.5809245	0.0000000	0.0000000	0.0000000	0.0000000
3.2772642	1.9900290	0.0000000	0.0000000	0.0000000
1.5704763	0.7303871	1.6905854	0.0000000	0.0000000
1.7127003	0.8495132	0.6884102	2.7178726	0.0000000
0.2226682	1.7758107	1.6682254	0.9340207	0.5878712

Table 10: HY matrix

4.5809245	0.0000000	0.0000000	0.0000000	0.0000000
3.2772642	1.9900290	0.0000000	0.0000000	0.0000000
1.5704763	0.7303871	1.6905854	0.0000000	0.0000000
1.7127003	0.8495132	0.6884102	2.7178726	0.0000000
0.2226682	1.7758107	1.6682254	0.9340207	0.5878712

Table 11: HC matrix

4.5809245	0.0000000	0.0000000	0.0000000	0.0000000
-----------	-----------	-----------	-----------	-----------

3.2772642	1.9900290	0.0000000	0.0000000	0.0000000
1.5704763	0.7303871	1.6905854	0.0000000	0.0000000
1.7127003	0.8495132	0.6884102	2.7178726	0.0000000
0.2226682	1.7758107	1.6682254	0.9340207	0.5878712

Table 12: Algorithm A matrix

4.5809245	0.0000000	0.0000000	0.0000000	0.0000000
3.2772642	1.9900290	0.0000000	0.0000000	0.0000000
1.5704763	0.7303871	1.6905854	0.0000000	0.0000000
1.7127003	0.8495132	0.6884102	2.7178726	0.0000000
0.2226682	1.7758107	1.6682254	0.9340207	0.5878712

The methods under comparison include:

- Numpy’s Cholesky Decomposition: Serves as the benchmark, calculating the Cholesky decomposition of the downdated matrix $LL^T + zz^T$ to obtain a lower triangular matrix.
- Specialized Downdating Algorithms: Adaptations of the HY, HC, and Algorithm A methods, now optimized for the context of lower triangular matrices. These variations aim to adjust the Cholesky factor while accommodating the subtraction of zz^T , thus maintaining the lower triangular structure.

Table 13: Numpy Cholesky matrix

4.5959589	0.0000000	0.0000000	0.0000000	0.0000000
3.3232001	2.0367762	0.0000000	0.0000000	0.0000000
1.6347010	0.8688074	1.7797292	0.0000000	0.0000000
1.7235722	0.8438101	0.6171486	2.737466	0.0000000
0.3341629	2.0260409	1.8835595	0.881099	0.7691109

Table 14: HY matrix

4.5959589	0.0000000	0.0000000	0.0000000	0.0000000
3.3232001	2.0367762	0.0000000	0.0000000	0.0000000
1.6347010	0.8688074	1.7797292	0.0000000	0.0000000
1.7235722	0.8438101	0.6171486	2.737466	0.0000000
0.3341629	2.0260409	1.8835595	0.881099	0.7691109

Table 15: HC matrix

4.5959589	0.0000000	0.0000000	0.0000000	0.0000000
3.3232001	2.0367762	0.0000000	0.0000000	0.0000000
1.6347010	0.8688074	1.7797292	0.0000000	0.0000000
1.7235722	0.8438101	0.6171486	2.737466	0.0000000
0.3341629	2.0260409	1.8835595	0.881099	0.7691109

Table 16: Algorithm A matrix

4.5959589	0.0000000	0.0000000	0.0000000	0.0000000
3.2665435	2.1264546	0.0000000	0.0000000	0.0000000
1.5653389	0.5339105	1.9644753	0.0000000	0.0000000
1.7070977	0.7238253	0.3411054	2.8289307	0.0000000
0.2219398	1.4815002	0.8482022	0.2903126	2.466646

The provided tables summarize the results of timing benchmarks for various Cholesky downdating algorithms, both for the downdating process alone and for the subsequent multiplication of the downdated matrix by a vector v . These benchmarks were conducted on matrices of sizes $n = 1000$ and $n = 5000$, evaluating the performance of the algorithms implemented in Cython. The specific methods benchmarked include:

- Numpy Cholesky: Using NumPy’s built-in Cholesky decomposition function to compute the Cholesky factor of the downdated matrix and, in the second benchmark, then multiplying this factor by v .
- HY: A Cython implementation of a hyperbolic downdating algorithm.
- HC: A Cython implementation of a downdating algorithm using hyperbolic cosine adjustments.
- Algorithm A: A Cython implementation of a generic downdating algorithm.

For both matrix sizes, the custom Cython implementations (HY, HC, Algorithm A) significantly outperformed the Numpy Cholesky approach in terms of execution time for the downdating operation. This performance difference becomes even more pronounced as the matrix size increases from $n = 1000$ to $n = 5000$.

Table 17: Timing Benchmark of Cholesky Downdate $DD^T = LL^T - zz^T$

n	Numpy: $D_{Cholesky}$	D_{HY}	D_{HC}	D_A
100	0.0000895	0.0000308	0.0000218	0.0000324
200	0.0003333	0.0001312	0.0000900	0.0001114
400	0.0019873	0.0005893	0.0003891	0.0004537
800	0.0195788	0.0019217	0.0011400	0.0016703
1600	0.0580574	0.0071094	0.0047709	0.0070625
3200	0.2615385	0.0309897	0.0228442	0.0326268

Table 18: Complexity ratio $w = Dv$. Structure case ($L = I$).

Numpy: $D_{Cholesky}$	D_{HY}	D_{HC}	D_A
3.7	4.3	4.1	3.4
6.0	4.5	4.3	4.1
9.9	3.3	2.9	3.7
3.0	3.7	4.2	4.2
4.5	4.4	4.8	4.6

Table 19: Timing Benchmark $w = Dv$

n	Numpy: $D_{Cholesky}v$	$D_{HY}v$	$D_{HC}v$	D_Av
100	0.0001074	0.0000415	0.0000241	0.0000289
200	0.0003291	0.0001179	0.0000783	0.0000949

n	Numpy: $D_{Cholesky}v$	$D_{HY}v$	$D_{HC}v$	D_Av
400	0.0014616	0.0004470	0.0002977	0.0003627
800	0.0070381	0.0021052	0.0012749	0.0014745
1600	0.0575613	0.0089113	0.0051467	0.0059966
3200	0.2736619	0.0387213	0.0200980	0.0231042

Table 20: Complexity ratio $w = Dv$. Structure case ($L = I$).

Numpy: $D_{Cholesky}v$	$D_{HY}v$	$D_{HC}v$	D_Av
3.1	2.8	3.3	3.3
4.4	3.8	3.8	3.8
4.8	4.7	4.3	4.1
8.2	4.2	4.0	4.1
4.8	4.3	3.9	3.9

3.4 Cholvec Algorithm

3.4.1 Uppertriangular case

This experiment is designed to illustrate the effectiveness of different algorithms for computing the product of a downdated Cholesky factor D with a vector v directly, without explicitly constructing the matrix D . The context is the downdating of a Cholesky decomposition in the cases requiring the modification of a positive definite matrix by subtracting the outer product of a vector z .

The primary goal is to compare various methods for performing this operation in terms of accuracy and efficiency. The methods compared are:

- Standard Cholesky Decomposition: The baseline method where D is explicitly computed using the Cholesky decomposition (using MKL) on the downdated matrix $R^T R - zz^T$, and then multiplied by $v_{Cholesky} = D_{Cholesky}v$.
- Specialized Downdating Algorithms: This method computes the product Dv , where D is the downdated Cholesky factor obtained through the HY, HC or Algorithm A downdating algorithm. The computation is performed without explicitly forming D .

The outcomes are presented in a table, comparing $v_{Cholesky}$, v_{HY} , v_{HC} and v_A to assess the accuracy of each method against the reference method (standard Cholesky decomposition). This comparison aims to highlight the numerical accuracy of the alternative methods.

Table 21: Downdated $w = Dv$, where $DD^T = R^T R - zz^T$ (Upper triangular case)

$v_{Cholesky}$	v_{HY}	v_{HC}	v_A
2.0956793	2.0956793	2.0956793	2.0956793
3.1788212	3.1788212	3.1788212	3.1788212
3.3760235	3.3760235	3.3760235	3.3760235
2.2775735	2.2775735	2.2775735	2.2775735
3.3470903	3.3470903	3.3470903	3.3470903
-0.5000196	-0.5000196	-0.5000196	-0.5000196
3.5345820	3.5345820	3.5345820	3.5345820
0.9808948	0.9808948	0.9808948	0.9808948
2.7400655	2.7400655	2.7400655	2.7400655
1.9825138	1.9825138	1.9825138	1.9825138

$v_{Cholesky}$	v_{HY}	v_{HC}	v_A
----------------	----------	----------	-------

3.4.2 Lowertriangular case

composite base detected

The outcomes are presented in a table, comparing $v_{Cholesky}$, v_{HY} , v_{HC} , v_A and $v_{lin\ chol}$ to assess the accuracy of each method against the reference method (standard Cholesky decomposition). This comparison aims to highlight the numerical accuracy of the alternative methods.

Table 22: Downdated $w = Dv$, where $DD^T = LL^T - zz^T$, Lower triangular case ($L = I$)

$v_{Cholesky}$	v_{HY}	v_{HC}	v_A	$v_{lin\ chol}$
0.1273271	0.1273271	0.1273271	0.1273271	0.1273271
0.1962485	0.1962485	0.1962485	0.1962263	0.1962263
0.2672940	0.2672940	0.2672940	0.2671611	0.2671611
0.3197869	0.3197869	0.3197869	0.3199423	0.3199423
0.3930224	0.3930224	0.3930224	0.3929032	0.3929032
0.4647309	0.4647309	0.4647309	0.4642872	0.4642872
-0.4646146	-0.4646146	-0.4646146	-0.4650213	-0.4650213
-0.4011751	-0.4011751	-0.4011751	-0.4013086	-0.4013086
-0.3359687	-0.3359687	-0.3359687	-0.3359623	-0.3359623
-0.2696634	-0.2696634	-0.2696634	-0.2696127	-0.2696127
-0.2027662	-0.2027662	-0.2027662	-0.2027265	-0.2027265

Table 23: Timing Benchmark $w = Dv$, Lower triangular case ($L = I$).

n	Numpy: $D_{Cholesky}v$	$D_{HY}v$	$w = [D_{HY}v]$	$w = [[D_{LinCholvec}v] : id(C)]$
100	0.0000965	0.0000412	0.0000256	0.0000226
200	0.0003142	0.0001453	0.0000790	0.0000423
400	0.0014050	0.0006094	0.0003662	0.0000964
800	0.0072045	0.0025868	0.0013919	0.0001853
1600	0.0550164	0.0123059	0.0053540	0.0003308
3200	0.2671443	0.0646315	0.0251041	0.0006191

Table 24: Complexity ratio $w = Dv$, Lower triangular case ($L = I$).

Numpy: $D_{Cholesky}v$	$D_{HY}v$	$w = [D_{HY}v]$	$w = [[D_{LinCholvec}v] : id(C)]$
3.3	3.5	3.1	1.9
4.5	4.2	4.6	2.3
5.1	4.2	3.8	1.9
7.6	4.8	3.8	1.8
4.9	5.3	4.7	1.9

Table 25: Timing Benchmark $w = Dv$, (The rightmost functions written in Cython and C), Structure case ($L = I$).

n	Numpy: $D_{Cholesky}v$	$w = D_{HY}v$	$w = [D_{HY}v] : id(Cthon)$	$w = [D_{HY}v] : id(C)$	$w = [[D_{LinCholvec}v] : id(C)]$	$w = [D_{HC}v] : id(Cthon)$
100	0.0000932	0.0000384	0.0000314	0.0005212	0.0000232	0.0000175
200	0.0003180	0.0001415	0.0000822	0.0012146	0.0000172	0.0000526
400	0.0014022	0.0005262	0.0002832	0.0025017	0.0000513	0.0001939
800	0.0075920	0.0029149	0.0012099	0.0052552	0.0001738	0.0007738
1600	0.0631204	0.0130220	0.0073328	0.0140499	0.0007223	0.0057312
3200	0.2678068	0.0642999	0.0260914	0.0273017	0.0027369	0.0124611
6400	1.7362058	0.3519338	0.1272402	0.0810131	0.0134902	0.0549140
12800	10.2578906	1.6815470	0.7017022	0.2332662	0.0480003	0.2017975
25600	90.1623854	7.1527789	3.5506140	0.8715287	0.2089625	0.8383850

Table 26: Complexity ratio $w = Dv$, (The functions written in C), Structure case ($L = I$).

Numpy: $D_{Cholesky}v$	$D_{HY}v$	$w = [D_{HY}v]$	$w = [D_{HY}v] : id(Cthon)$	$w = [D_{HY}v] : id(C)$	$w = [[D_{LinCholvec}v] : id(C)]$	$w = [D_{HC}v] : id(Cthon)$
3.4	3.7	2.6	2.3	0.7	2.3	3.0
4.4	3.7	3.4	2.1	3.0	1.9	3.7
5.4	5.5	4.3	2.1	3.4	2.0	4.0
8.3	4.5	6.1	2.7	4.2	2.2	7.4
4.2	4.9	3.6	1.9	3.8	1.8	2.2
6.5	5.5	4.9	3.0	4.9	2.1	4.4
5.9	4.8	5.5	2.9	3.6	1.9	3.7
8.8	4.3	5.1	3.7	4.4	2.1	4.2

Table 27: Timing Benchmark $w = Dv$, (The functions written in C), Structure case ($L = I$). For $n = 10000...160000$

n	$w = [D_{HY}v] : id(C)$	$w = [D_{HY}v] : id(C)vec$	$w = [D_{alg}Av] : id(C)$	$w = [[D_{HC}v] : id(C)]$	$w = [[D_{LinCholvec}v] : id(C)]$
10000	0.0371661	0.0462044	0.0333661	0.0521230	0.0030777
20000	0.1477311	0.1853011	0.1329403	0.2133342	0.0060857
40000	0.6088186	0.7443512	0.5923786	0.9410146	0.0097728
80000	2.5745531	3.1632138	2.3124696	3.8921592	0.0231563
160000	9.9178213	12.1740603	10.0497361	13.8990540	0.0447739

Table 28: Complexity ratio $w = Dv$, (The functions written in C), Structure case ($L = I$). For $n = 10000...160000$

$w = [D_{HY}v] : id(C)$	$w = [D_{HY}v] : id(C)vec$	$w = [D_{alg}Av] : id(C)$	$w = [[D_{HC}v] : id(C)]$	$w = [[D_{LinCholvec}v] : id(C)]$
4.0	4.0	4.0	4.1	2.0
4.1	4.0	4.5	4.4	1.6

$w = [D_{HY}v] : id(C)$	$w = [D_{HY}v] : id(C)vec$	$w = [D_{alg}Av] : id(C)$	$w = [[D_{HC}v] : id(C)$	$w = [[D_{LinCholvec}v] : id(C)$
4.2	4.2	3.9	4.1	2.4
3.9	3.8	4.3	3.6	1.9

In the benchmarking experiment, various Cholesky decomposition and downdating methods are tested across a range of matrix sizes to evaluate their performance. The experiment includes a baseline implementation using NumPy’s Cholesky function (Numpy: $D_{Cholesky}v$), several custom Cholesky update implementations ($D_{HY}v$, $w = [D_{HY}v]$), and optimizations that incorporate Cython and C implementations with different optimization strategies ($w = [D_{HY}v] : id(C_{thon})$, $w = [D_{HY}v] : id(C)$, $w = [D_{HY}v] : id(C)vec$). Each method is executed 10 times for matrices of sizes ranging from 100 to 20,000, with the average execution time recorded to compare the efficiency of downdating processes across different algorithmic approaches and implementations.

3.5 Rank M Cholesky Factor Update

$$D = R + ZZ^T = R + \sum_{i=1}^M z_i z_i^T = ((R + z_1 z_1^T) + \dots) + \sum_{i=k}^M z_i z_i^T$$

It has the complexity $O(n^2 M)$.

In this experiment, we compare the performance of iterative Cholesky updates using NumPy’s built-in Cholesky decomposition against a custom hyperbolic Cholesky update algorithm (referred to as “HY”). The objective is to evaluate the efficiency of applying a series of rank-one updates to the Cholesky factor of a large positive definite matrix.

The setup for our experiment is as follows:

1. A square positive definite matrix A of size $n = 1000$ is generated. This is accomplished by computing $A = ZZ^T + nI$, where Z is a randomly generated matrix, and I is the identity matrix. This construction ensures that A is positive definite.
2. A set of update vectors is created by selecting the first 100 columns of A , denoted as the update factor matrix.

The results of the experiment are summarized in the following table:

References

- [1] Chambers, J. M. (1971). Regression Updating. Journal of the American Statistical Association, 66(336), 744–748. <https://doi.org/10.2307/2284221>
- [2] Chi-Tsuan Pan, Thesis
- [3] Hyperbolic rotations